

1. Python

</> OneCompiler

Upgrade

AI

Python

Run

:

🔍

⚙️

Save

🔗

Login

main.py

+

1 def adicionar(item, lista=[]):
2 lista.append(item)
3 return lista
4
5 print(adicionar(1))
6 print(adicionar(2))

Console

I/O

AI Agent

132 ms

✓

[1]
[1, 2]

Ready

> Terminal

Light

Wiki

Internet

2. Java

</> OneCompiler

Upgrade

AI

Java

Run

:

🔍

⚙️

Save

🔗

Login

Main.java

+

1 public class Main {
2
3 static void zera(int[] v, int n) {
4 v[0] = 0;
5 n = 0;
6 }
7
8 public static void main(String[] args) {
9 int[] v = {5, 5};
10 int n = 5;
11
12 zera(v, n);
13
14 System.out.println(v[0] + " " + n);
15 }
16 }

Console

I/O

AI Agent

826 ms

✓

0 5

Ready

> Terminal

Light

Wiki

Internet

3. Python

OneCompiler

UpgradeAI

Python

Run

:

Save

Login

main.py

+

1 fs = [lambda: i for i in range(3)]

2

3 print([f() for f in fs])

Console

I/O

AI Agent

145 ms

[2, 2, 2]

Ready

Terminal

Light

Wiki

Internet

4. C (main completo)

OneCompiler

UpgradeAI

C

Run

:

Save

Login

C Main.c

+

1 #include <stdio.h>

2

3 int contador(void) {

4 static int n = 0;

5 return ++n;

6 }

7

8 int main(void) {

9 contador();

10 contador();

11

12 printf("%d\n", contador());

13

14 return 0;

15 }

16

17

Console

I/O

AI Agent

174 ms

3

Ready

Terminal

Light

Wiki

Internet

5. Rust

```
fn dobra(v: &Vec<i32>)  
dobra(&v)
```

OneCompiler

UpgradeAI

Rust

Run

:

Save

Login

HelloWorld.rs

1 fn dobra(v: &Vec<i32>) -> Vec<i32> {
2 v.iter().map(|x| x * 2).collect()
3 }
4
5 fn main() {
6 let v = vec![1, 2, 3];
7 let d = dobra(&v);
8
9 println!("{:?} {:?}", v, d);
10 }

Console

I/O

AI Agent

264 ms

[1, 2, 3] [2, 4, 6]

Ready

LightWikiInternet

6. Python

global total

OneCompiler

UpgradeAI

Python

Run

:

Save

Login

main.py

1 total = 0
2
3 def adiciona(x):
4 global total
5 total = total + x
6 return total
7
8 print(adiciona(5))

Console

I/O

AI Agent

126 ms

5

Ready

Terminal

LightWikiInternet